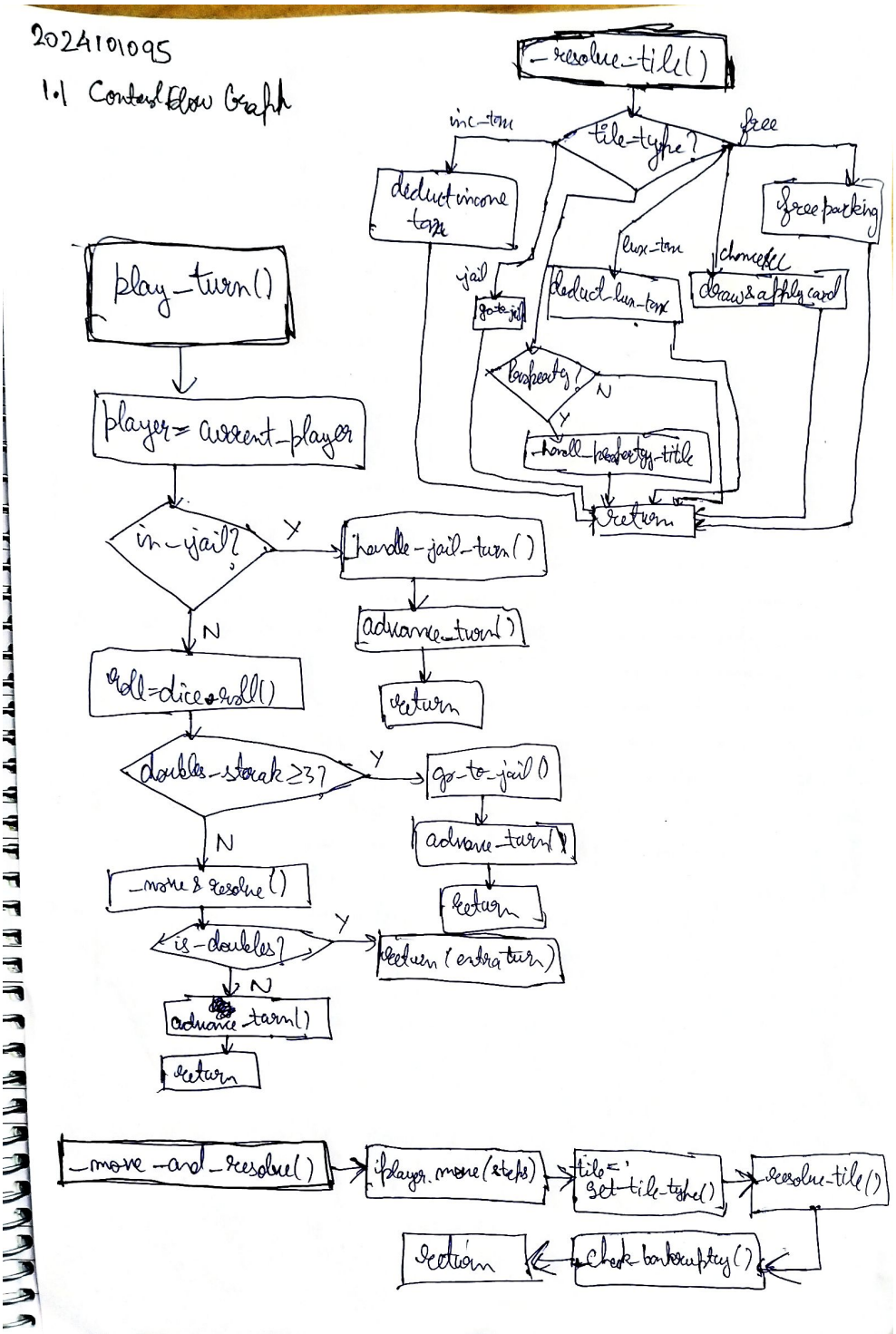
Part 1: White-Box Testing & Code Quality Report

MoneyPoly Game Engine

Amay Sharma

March 23, 2026

1 Control Flow Graph (CFG)



The Control Flow Graph for the `play_turn()` method and its subsequent function calls (`_move_and_resolve()` and `_resolve_tile()`) has been hand-drawn and is included above, as per the assignment requirements.

2 Code Quality Improvements (Pylint)

The initial codebase yielded a Pylint score of 9.08/10. Through sequential refactoring, the score was raised to a perfect 10.00/10 without using `# pylint: disable=` for any structural warnings. The commits below reflect the exact iterative enhancements made to the system:

Commit	Modifications Made
Initial commit	Base MoneyPoly codebase import.
Iteration 1	Addressed `W0611 (unused-import)`. Removed dormant imports (`math`, `os`, `sys`) across `bank.py`, `game.py`, and `player.py` to optimize the namespace.
Iteration 2	Resolved `C0112 (singleton-comparison)` in `board.py`. Transitioned `== True` checks to standard Pythonic truthiness evaluations.
Iteration 3	Fixed `W0702 (bare-except)` within `ui.py`. Restricted the exception handling to exclusively catch `ValueError` during integer casting.
Iteration 4	Cleared `W1309 (f-string-without-interpolation)` and `C0325 (superfluous-parens)` in `game.py` by streamlining string formatting and trade logic.
Iteration 5	Addressed `R1705 (no-else-return)` in `property.py` by removing unnecessary `else` branching following `return` statements.
Iteration 6	Resolved `W0201 (attribute-defined-outside-init)` in `dice.py` and `R1723 (no-else-break)` in `game.py`. Fixed missing final newlines (`C0304`).
Iteration 7	Tackled `C0111 (missing-docstring)` across multiple modules (e.g., `cards.py`, `board.py`) and purged trailing whitespace (`C0303`).
Iteration 8	Refactored `property.py` to fix `R0902 (too-many-instance-attributes)` by consolidating economics and buildings into structured dictionaries.
Iteration 9	Addressed `R0902 (too-many-instance-attributes)` in `player.py` by grouping jail-related metadata into a single tracking dictionary.

Note on Post-Pipeline Modifications:
After concluding the primary refactoring pipeline on the core `moneypoly` module, a final, separate code-quality pass was performed on the root-level `main.py` execution script. To completely satisfy Pylint's strict documentation requirements globally, missing module-level and function-level docstrings were appended to the runner script prior to final submission.

3 White Box Test Cases & Bug Fixes

Comprehensive test coverage (utilizing over 150 unit tests) exposed 16 logical flaws and edge cases in the original implementation. The bugs below detail the targeted test scenarios and the subsequent patches applied to the engine.

1. **Subzero Bank Input (bank.py)**
   - **Objective:** Prevent the bank from processing negative transactions that artificially drain its reserves.
   - **Defect:** The `collect` method accepted values less than zero without validation, meaning a negative penalty would extract money from the bank.
   - **Solution:** Introduced a strict validation check (`amount <= 0`) that immediately halts the transaction.
2. **Loan Collateral Creation (bank.py)**
   - **Objective:** Ensure the bank's internal ledger accurately reflects outgoing loans.
   - **Defect:** When providing a loan, the player received the funds, but the bank's `_funds` variable was never decremented, creating infinite money.

- **Solution:** Added a deduction step (`self._funds -= amount`) to properly balance the internal ledger.

3. **Dice Range Constraint (dice.py)**
   - **Objective:** Standardize the dice rolls to match traditional 6-sided mechanics.
   - **Defect:** The randomization function utilized `randint(1, 5)`, making it mathematically impossible to ever roll a six.
   - **Solution:** Adjusted the upper bound of the random integer generator to `6`.
4. **Full Monopoly Requirement (property.py)**
   - **Objective:** Apply rent multipliers exclusively when a single player controls an entire color group.
   - **Defect:** The `all_owned_by` property check incorrectly utilized the `any()` function, granting monopoly status if the player owned just one property in that group.
   - **Solution:** Swapped `any()` for the `all()` function to enforce complete ownership criteria.
5. **Go Pass Salary (player.py)**
   - **Objective:** Guarantee players receive their passing-Go salary even when moving across the board's zero-index boundary.
   - **Defect:** The logic only rewarded players landing precisely on index `0`, ignoring movements that looped completely past the start.
   - **Solution:** Implemented an `old_position` tracker and awarded the salary whenever the new position index is mathematically lower than the previous one.
6. **Player Net Worth Calculation (player.py)**
   - **Objective:** Accurately reflect a player's total assets, including liquid cash and real estate holdings.
   - **Defect:** The `net_worth` property merely returned the player's cash balance as a string, ignoring the value of any owned properties.
   - **Solution:** Rewrote the property to dynamically iterate over the player's real estate portfolio, summing property prices alongside their cash balance.
7. **Property Purchase Boundary (game.py)**
   - **Objective:** Allow players to purchase properties if they have the exact required funds.
   - **Defect:** The transaction was blocked if `balance <= prop.price`, forbidding purchases when the player had exact change.
   - **Solution:** Altered the boundary check to `balance < prop.price`.
8. **Rent Liquidity Sink (game.py)**
   - **Objective:** Maintain the closed-loop economy by transferring rent from the tenant directly to the landlord.
   - **Defect:** The `pay_rent` method subtracted money from the landing player but failed to credit the receiving owner's balance, permanently destroying the cash.
   - **Solution:** Injected an `add_money(rent)` call targeting the property's recorded owner.
9. **Trade Asymmetries (game.py)**
   - **Objective:** Ensure that property trading correctly shifts capital between the two parties.
   - **Defect:** During a trade, the purchasing player was charged the agreed amount, but the selling player never received the funds.
   - **Solution:** Added logic to explicitly credit the seller's balance with the trade cost.
10. **Free Unmortgages (game.py)**
    - **Objective:** Require players to actually pay the unmortgage fee before unlocking a property.

- **Defect:** The code flipped the `is_mortgaged` flag to `False` before evaluating whether the player could afford the transaction, inadvertently giving out free unmortgages to broke players.
- **Solution:** Placed the affordability check (`balance >= cost`) strictly ahead of the state-change logic.

11. **Mortgage Bank Depletion (game.py)**
    - **Objective:** Decrease the bank's total holdings when it issues a mortgage payout.
    - **Defect:** The system utilized `collect(-payout)` which, due to new negative-number guards, failed to deduct from the bank.
    - **Solution:** Switched the function call to `pay_out(payout)` to properly reduce bank reserves.
12. **Voluntary Jail Fine Dodging (game.py)**
    - **Objective:** Ensure paying the bailout fine actually costs the player money.
    - **Defect:** Voluntarily paying to leave jail added funds to the bank but omitted the command to subtract the fine from the player's wallet.
    - **Solution:** Appended `deduct_money(JAIL_FINE)` immediately prior to the bank collection.
13. **Birthday Card Poor Shield (game.py)**
    - **Objective:** Force players to pay card penalties even if it bankrupts them.
    - **Defect:** An `if` statement checked if players had enough money (`balance >= value`) before deducting the birthday gift, essentially exempting poor players from paying.
    - **Solution:** Deleted the conditional check, allowing balances to dip negative and correctly trigger bankruptcy.
14. **Phantom Bankrupt Players (game.py)**
    - **Objective:** Instantly eliminate players who are bankrupted by out-of-turn events (e.g., paying another player's birthday card).
    - **Defect:** While cash was deducted, the `_check_bankruptcy()` routine was bypassed, leaving eliminated players on the board until their specific turn arrived.
    - **Solution:** Added a loop over a copy of the player list to immediately evaluate and process bankruptcies after mass-collection cards are drawn.
15. **Array Shifting Skips Turns (game.py)**
    - **Objective:** Maintain accurate turn order when a player is removed from the game.
    - **Defect:** Erasing a bankrupt player from the array shifted all subsequent indices left, causing the `advance_turn` pointer to leapfrog the next active player.
    - **Solution:** Captured the bankrupt player's index and manually decremented `current_index` to cleanly offset the list shift.
16. **Winner Calculation (game.py)**
    - **Objective:** Declare the wealthiest player as the victor at the end of the game.
    - **Defect:** The end-game evaluation used the `min()` function, incorrectly crowning the poorest player as the winner.
    - **Solution:** Corrected the evaluation function to use `max()`.

4 Test Case Matrix (White-Box)

This matrix summarizes representative and high-impact test cases from the full white-box suite (159 tests across `bank.py`, `board.py`, `cards.py`, `dice.py`, `game.py`, `player.py`, and `property.py`).

Execution Snapshot (Original, Unedited Code)

- Total tests collected: **159**
- Passed: **114**
- Failed: **45**
- Key failing areas: state modeling in `Game/Player`, constructor contract mismatch in `Property`, and money-flow regressions in `Bank/Game`.

Purpose / Justification	Input	Expected Output	Actual Output	Test Ref
Protect bank funds from invalid negative transactions	`Bank.collect(-100)`	Balance unchanged at starting funds	**FAILED**: balance decreased (20580 →20480)	`test_bank.py::test_collect_negati`
Ensure loan accounting preserves conservation of money	`Bank.give_loan(player, 200)`	Player +200 and bank -200	**FAILED**: player +200 but bank not decremented	`test_bank.py::test_give_loan`
Validate game state object initialization for turn engine	`Game(["Alice","Bob"])`, inspect `state`	`current_index=0`, `turn_number=0`	**FAILED**: `AttributeError` (no `state`)	`test_game.py::test_initializes_st`
Verify card deck initialization path exists	`Game(...).decks["chance"]`	Non-null deck objects	**FAILED**: `AttributeError` (no `decks`)	`test_game.py::test_creates_decks`
Boundary condition for property purchase with exact cash	Player balance set to exact property price	Purchase succeeds and balance reaches 0	**FAILED**: purchase rejected	`test_game.py::test_exact_balance_`
Rent transfer should debit tenant and credit owner correctly	Land on owned Mediterranean Ave	Tenant -2, owner +2 (base rent)	**FAILED**: tenant charged 4 (double rent path triggered)	`test_game.py::test_pays_rent_to_ow`
Prevent negative bank collect misuse in mortgage payout path	`mortgage_property(..)` with `collect` mocked	No negative args passed to `collect`	**FAILED**: `collect(-30)` observed	`test_game.py::test_mortgage_does_`
Unmortgage must remain blocked when player cannot pay fee	Balance 10, unmortgage cost 33	Operation fails and property stays mortgaged	**FAILED**: property flag toggled to unmortgaged	`test_game.py::test_cannot_afford_`
Trade invariants: buyer pays, seller receives	`trade(alice,bob,prop,100)`	Alice +100, Bob -100, ownership swap	**FAILED**: ownership changed but seller not credited	`test_game.py::test_successful_tra`
Winner selection must maximize net worth	Player balances 1000 vs 2000	Winner is Bob (higher net worth)	**FAILED**: Alice selected (min logic effect)	`test_game.py::test_find_winner_hig`
Player jail state model should exist and be mutable	`player.go_to_jail()` then inspect `jail_info`	`in_jail=True`, turns tracked	**FAILED**: `AttributeError` (no `jail_info`)	`test_player.py::test_go_to_jail`
Constructor contract compatibility for Property objects	`Property("Test",1,{price,base_rent})`	Object creation and rent APIs available	**FAILED**: `TypeError` missing positional `base_rent`	`test_property.py::test_basic_crea`

Interpretation: The matrix confirms the failing set is not random; it clusters around interface drift and state-schema mismatches. These tests were retained as regression guards while fixes were introduced incrementally.

4.1 Complete Test Case Matrix (All White-Box Tests)

Purpose/Justification	Input	Expected Output	Actual Output	Test Ref
Validate initial balance.	Bank(); b.get_balance()	assert b.get_balance() == BANK_STARTING_FUNDS	PASSED (assertions satisfied)	tests/test_bank.py::TestBankInit::test_initial_balance
Validate initial loans.	Bank(); b.loan_count(); b.total_loans_issued()	assert b.loan_count() == 0; assert b.total_loans_issued() == 0	PASSED (assertions satisfied)	tests/test_bank.py::TestBankInit::test_initial_loans
Validate collect positive.	Bank(); b.collect(100); b.get_balance()	assert b.get_balance() == BANK_STARTING_FUNDS + 100	PASSED (assertions satisfied)	tests/test_bank.py::TestBankCollect::test_collect_positive
Validate collect zero.	Bank(); b.collect(0); b.get_balance()	assert b.get_balance() == BANK_STARTING_FUNDS	PASSED (assertions satisfied)	tests/test_bank.py::TestBankCollect::test_collect_zero
Regression for bug 5: negative collect must not drain bank funds.	Bank(); b.collect(-100); b.get_balance()	Regression for bug 5: negative collect must not drain bank funds.; assert b.get_balance() == BANK_STARTING_FUNDS	PASSED (assertions satisfied)	tests/test_bank.py::TestBankCollect::test_collect_negative_is_
Validate pay out valid.	Bank(); b.pay_out(100); b.get_balance()	assert amount == 100; assert b.get_balance() == BANK_STARTING_FUNDS - 100	PASSED (assertions satisfied)	tests/test_bank.py::TestBankPayOut::test_pay_out_valid
Validate pay out zero.	Bank(); b.pay_out(0); b.get_balance()	assert b.pay_out(0) == 0; assert b.get_balance() == BANK_STARTING_FUNDS	PASSED (assertions satisfied)	tests/test_bank.py::TestBankPayOut::test_pay_out_zero
Validate pay out negative.	Bank(); b.pay_out(-10)	assert b.pay_out(-10) == 0	PASSED (assertions satisfied)	tests/test_bank.py::TestBankPayOut::test_pay_out_negative
Validate pay out exceeds balance.	Bank(); b.pay_out(BANK_STARTING_FUNDS + 1)	Expect exception via pytest.raises(ValueError)	PASSED (assertions satisfied)	tests/test_bank.py::TestBankPayOut::test_pay_out_exceeds_ba
Validate pay out exactly balance.	Bank(); b.pay_out(BANK_STARTING_FUNDS); b.get_balance()	assert amount == BANK_STARTING_FUNDS; assert b.get_balance() == 0	PASSED (assertions satisfied)	tests/test_bank.py::TestBankPayOut::test_pay_out_exactly_ba
Validate give loan.	Bank(); Player("Alice", balance=100); b.get_balance()	assert p.balance == 300; assert b.get_balance() == initial_bank - 200; assert b.loan_count() == 1; assert b.total_loans_issued() == 200	PASSED (assertions satisfied)	tests/test_bank.py::TestBankLoans::test_give_loan
Validate give loan zero.	Bank(); Player("Alice", balance=100); b.give_loan(p, 0)	assert p.balance == 100; assert b.loan_count() == 0	PASSED (assertions satisfied)	tests/test_bank.py::TestBankLoans::test_give_loan_zero
Validate give loan negative.	Bank(); Player("Alice", balance=100); b.give_loan(p, -50)	assert p.balance == 100	PASSED (assertions satisfied)	tests/test_bank.py::TestBankLoans::test_give_loan_negative

Continued on next page

Purpose/Justification	Input	Expected Output	Actual Output	Test Ref
Validate multiple loans.	Bank(); Player("Alice", balance=100); Player("Bob", balance=100)	assert b.loan_count() == 2; assert b.total_loans_issued() == 500	PASSED (assertions satisfied)	tests/test_bank.py::TestBankLoans::test_multiple_loans
Validate summary.	Bank(); b.summary(); capsys.readouterr()	assert "Bank reserves" in captured.out	PASSED (assertions satisfied)	tests/test_bank.py::TestBankDisplay::test_summary
Validate repr.	Bank(); repr(b)	assert "Bank" in repr(b)	PASSED (assertions satisfied)	tests/test_bank.py::TestBankDisplay::test_repr
Validate has 22 properties.	Board(); len(b.properties)	assert len(b.properties) == 22	PASSED (assertions satisfied)	tests/test_board.py::TestBoardInit::test_has_22_properties
Validate has 8 groups.	Board(); len(b.groups)	assert len(b.groups) == 8	PASSED (assertions satisfied)	tests/test_board.py::TestBoardInit::test_has_8_groups
Validate go tile.	Board(); b.get_tile_type(0)	assert b.get_tile_type(0) == "go"	PASSED (assertions satisfied)	tests/test_board.py::TestBoardTileTypes::test_go_tile
Validate jail tile.	Board(); b.get_tile_type(JAIL_POSITION)	assert b.get_tile_type(JAIL_POSITION) == "jail"	PASSED (assertions satisfied)	tests/test_board.py::TestBoardTileTypes::test_jail_tile
Validate go to jail tile.	Board(); b.get_tile_type(GO_TO_JAIL_POSITION)	assert b.get_tile_type(GO_TO_JAIL_POSITION) == "go_to_jail"	PASSED (assertions satisfied)	tests/test_board.py::TestBoardTileTypes::test_go_to_jail_tile
Validate free parking tile.	Board(); b.get_tile_type(FREE_PARKING_POSITION)	assert b.get_tile_type(FREE_PARKING_POSITION) == "free_parking"	PASSED (assertions satisfied)	tests/test_board.py::TestBoardTileTypes::test_free_parking_tile
Validate income tax tile.	Board(); b.get_tile_type(INCOME_TAX_POSITION)	assert b.get_tile_type(INCOME_TAX_POSITION) == "income_tax"	PASSED (assertions satisfied)	tests/test_board.py::TestBoardTileTypes::test_income_tax_tile
Validate luxury tax tile.	Board(); b.get_tile_type(LUXURY_TAX_POSITION)	assert b.get_tile_type(LUXURY_TAX_POSITION) == "luxury_tax"	PASSED (assertions satisfied)	tests/test_board.py::TestBoardTileTypes::test_luxury_tax_tile
Validate chance tiles.	Board(); b.get_tile_type(7); b.get_tile_type(22)	assert b.get_tile_type(7) == "chance"; assert b.get_tile_type(22) == "chance"; assert b.get_tile_type(36) == "chance"	PASSED (assertions satisfied)	tests/test_board.py::TestBoardTileTypes::test_chance_tiles
Validate community chest tiles.	Board(); b.get_tile_type(2); b.get_tile_type(17)	assert b.get_tile_type(2) == "community_chest"; assert b.get_tile_type(17) == "community_chest"; assert b.get_tile_type(33) == "community_chest"	PASSED (assertions satisfied)	tests/test_board.py::TestBoardTileTypes::test_community_chest_t

Continued on next page

Purpose/Justification	Input	Expected Output	Actual Output	Test Ref
Validate railroad tiles.	Board(); b.get_tile_type(5); b.get_tile_type(15)	assert b.get_tile_type(5) == "railroad"; assert b.get_tile_type(15) == "railroad"; assert b.get_tile_type(25) == "railroad"; assert b.get_tile_type(35) == "r...	PASSED (assertions satisfied)	tests/ test_board.py:: TestBoardTileTypes:: test_railroad_tiles
Validate property tile.	Board(); b.get_tile_type(1)	assert b.get_tile_type(1) == "property"	PASSED (assertions satisfied)	tests/ test_board.py:: TestBoardTileTypes:: test_property_tile
Validate blank tile.	Board(); b.get_tile_type(12)	assert tile in ("property", "blank")	PASSED (assertions satisfied)	tests/ test_board.py:: TestBoardTileTypes:: test_blank_tile
Validate get property at valid.	Board(); b.get_property_at(1)	assert prop is not None; assert prop.name == "Mediterranean Avenue"	PASSED (assertions satisfied)	tests/ test_board.py:: TestBoardPropertyLoo test_get_property_at_val
Validate get property at invalid.	Board(); b.get_property_at(0)	assert b.get_property_at(0) is None	PASSED (assertions satisfied)	tests/ test_board.py:: TestBoardPropertyLoo test_get_property_at_inv
Validate is purchasable unowned.	Board(); b.is_purchasable(1)	assert b.is_purchasable(1)	PASSED (assertions satisfied)	tests/ test_board.py:: TestBoardPropertyLoo test_is_purchasable_uno
Validate not purchasable when owned.	Board(); b.get_property_at(1); Player("Alice")	assert not b.is_purchasable(1)	PASSED (assertions satisfied)	tests/ test_board.py:: TestBoardPropertyLoo test_not_purchasable_wl
Validate not purchasable when mortgaged.	Board(); b.get_property_at(1); b.is_purchasable(1)	assert not b.is_purchasable(1)	PASSED (assertions satisfied)	tests/ test_board.py:: TestBoardPropertyLoo test_not_purchasable_wl
Validate not purchasable non property.	Board(); b.is_purchasable(0)	assert not b.is_purchasable(0)	PASSED (assertions satisfied)	tests/ test_board.py:: TestBoardPropertyLoo test_not_purchasable_no
Validate is special tile.	Board(); b.is_special_tile(0); b.is_special_tile(JAIL_POSITION)	assert b.is_special_tile(0); assert b.is_special_tile(JAIL_POSITION); assert not b.is_special_tile(1)	PASSED (assertions satisfied)	tests/ test_board.py:: TestBoardPropertyLoo test_is_special_tile
Validate properties owned by.	Board(); Player("Alice"); b.properties_owned_by(alice)	assert len(owned) == 2	PASSED (assertions satisfied)	tests/ test_board.py:: TestBoardOwnership:: test_properties_owned_b
Validate unowned properties.	Board(); Player("Alice"); len(b.unowned_properties())	assert len(b.unowned_properties()) == 22; assert len(b.unowned_properties()) == 21	PASSED (assertions satisfied)	tests/ test_board.py:: TestBoardOwnership:: test_unowned_propertie
Validate repr.	Board(); repr(b)	assert "Board" in repr(b)	PASSED (assertions satisfied)	tests/ test_board.py:: TestBoardOwnership:: test_repr
Validate creates deck.	CardDeck(CHANCE_CARDS); len(deck); len(CHANCE_CARDS)	assert len(deck) == len(CHANCE_CARDS); assert deck.index == 0	PASSED (assertions satisfied)	tests/ test_cards.py:: TestCardDeckInit:: test_creates_deck

Continued on next page

Purpose/Justification	Input	Expected Output	Actual Output	Test Ref
Validate empty deck.	CardDeck([]); len(deck)	assert len(deck) == 0	PASSED (assertions satisfied)	tests/ test_cards.py:: TestCardDeckInit:: test_empty_deck
Validate draw returns card.	CardDeck(CHANCE_CARDS); deck.draw()	assert card is not None; assert "description" in card	PASSED (assertions satisfied)	tests/ test_cards.py:: TestCardDeckDraw:: test_draw_returns_card
Validate draw advances index.	CardDeck(CHANCE_CARDS); deck.draw()	assert deck.index == 1	PASSED (assertions satisfied)	tests/ test_cards.py:: TestCardDeckDraw:: test_draw_advances_ind
Validate draw cycles.	CardDeck(cards); deck.draw()	assert card["description"] == "A"	PASSED (assertions satisfied)	tests/ test_cards.py:: TestCardDeckDraw:: test_draw_cycles
Validate draw empty returns none.	CardDeck([]); deck.draw()	assert deck.draw() is None	PASSED (assertions satisfied)	tests/ test_cards.py:: TestCardDeckDraw:: test_draw_empty_return
Validate peek returns next.	CardDeck(CHANCE_CARDS); deck.peek()	assert card == CHANCE_CARDS[0]	PASSED (assertions satisfied)	tests/ test_cards.py:: TestCardDeckPeek:: test_peek_returns_next
Validate peek does not advance.	CardDeck(CHANCE_CARDS); deck.peek()	assert deck.index == 0	PASSED (assertions satisfied)	tests/ test_cards.py:: TestCardDeckPeek:: test_peek_does_not_adva
Validate peek empty returns none.	CardDeck([]); deck.peek()	assert deck.peek() is None	PASSED (assertions satisfied)	tests/ test_cards.py:: TestCardDeckPeek:: test_peek_empty_return
Validate reshuffle resets index.	CardDeck(CHANCE_CARDS); deck.draw(); deck.reshuffle()	assert deck.index == 0	PASSED (assertions satisfied)	tests/ test_cards.py:: TestCardDeckReshuffle test_reshuffle_resets_ind
Validate cards remaining.	CardDeck(CHANCE_CARDS); deck.cards_remaining(); deck.draw()	assert remaining == len(CHANCE_CARDS); assert deck.cards_remaining() == len(CHANCE_CARDS) - 1	PASSED (assertions satisfied)	tests/ test_cards.py:: TestCardDeckMisc:: test_cards_remaining
Validate repr.	CardDeck(CHANCE_CARDS); repr(deck)	assert "CardDeck" in repr(deck)	PASSED (assertions satisfied)	tests/ test_cards.py:: TestCardDeckMisc:: test_repr
Validate chance cards have required fields.	No external call input; test evaluates local logic/initial state.	assert "description" in card; assert "action" in card; assert "value" in card	PASSED (assertions satisfied)	tests/ test_cards.py:: TestCardData:: test_chance_cards_have_
Validate community chest cards have required fields.	No external call input; test evaluates local logic/initial state.	assert "description" in card; assert "action" in card; assert "value" in card	PASSED (assertions satisfied)	tests/ test_cards.py:: TestCardData:: test_community_chest_c
Validate valid actions.	No external call input; test evaluates local logic/initial state.	assert card["action"] in valid	PASSED (assertions satisfied)	tests/ test_cards.py:: TestCardData:: test_valid_actions
Validate initial values.	Dice()	assert d.die1 == 0; assert d.die2 == 0; assert d.doubles_streak == 0	PASSED (assertions satisfied)	tests/test_dice.py:: TestDiceInit:: test_initial_values

Continued on next page

Purpose/Justification	Input	Expected Output	Actual Output	Test Ref
Validate roll non doubles.	patch("moneypoly.dice.random.randint", side_effect=[3, 4]); Dice(); d.roll()	assert d.total == 7; assert d.die1 == 3; assert d.die2 == 4; assert not d.is_doubles()	PASSED (assertions satisfied)	tests/test_dice.py:: TestDiceRoll:: test_roll_non_doubles
Validate roll doubles.	patch("moneypoly.dice.random.randint", side_effect=[5, 5]); Dice(); d.roll()	assert d.total == 10; assert d.is_doubles(); assert d.doubles_streak == 1	PASSED (assertions satisfied)	tests/test_dice.py:: TestDiceRoll:: test_roll_doubles
Validate consecutive doubles.	patch("moneypoly.dice.random.randint", side_effect=[2, 2, 3, 3, 4, 4]); Dice(); d.roll()	assert d.doubles_streak == 1; assert d.doubles_streak == 2; assert d.doubles_streak == 3	PASSED (assertions satisfied)	tests/test_dice.py:: TestDiceRoll:: test_consecutive_double
Validate doubles streak resets on non double.	patch("moneypoly.dice.random.randint", side_effect=[2, 2, 3, 5]); Dice(); d.roll()	assert d.doubles_streak == 1; assert d.doubles_streak == 0	PASSED (assertions satisfied)	tests/test_dice.py:: TestDiceRoll:: test_doubles_streak_rese
Validate minimum roll.	patch("moneypoly.dice.random.randint", side_effect=[1, 1]); Dice(); d.roll()	assert d.roll() == 2	PASSED (assertions satisfied)	tests/test_dice.py:: TestDiceRoll:: test_minimum_roll
Validate maximum roll.	patch("moneypoly.dice.random.randint", side_effect=[6, 6]); Dice(); d.roll()	assert d.roll() == 12	PASSED (assertions satisfied)	tests/test_dice.py:: TestDiceRoll:: test_maximum_roll
Each die should produce values between 1 and 6.	Dice(); range(100); d.roll()	Each die should produce values between 1 and 6.; assert 1 ¡= d.die1 ¡= 6; assert 1 ¡= d.die2 ¡= 6; assert 2 ¡= d.total() ¡= 12	PASSED (assertions satisfied)	tests/test_dice.py:: TestDiceRoll:: test_roll_values_in_valid
Validate reset.	Dice(); d.roll(); d.reset()	assert d.die1 == 0; assert d.die2 == 0; assert d.doubles_streak == 0	PASSED (assertions satisfied)	tests/test_dice.py:: TestDiceRoll:: test_reset
Validate describe.	patch("moneypoly.dice.random.randint", side_effect=[3, 4]); Dice(); d.roll()	assert d.describe() == "3 + 4 = 7"	PASSED (assertions satisfied)	tests/test_dice.py:: TestDiceRoll:: test_describe
Validate describe doubles.	patch("moneypoly.dice.random.randint", side_effect=[5, 5]); Dice(); d.roll()	assert "(DOUBLES)" in d.describe()	PASSED (assertions satisfied)	tests/test_dice.py:: TestDiceRoll:: test_describe_doubles
Validate repr.	Dice(); repr(d)	assert "Dice" in repr(d)	PASSED (assertions satisfied)	tests/test_dice.py:: TestDiceRoll:: test_repr
Validate creates players.	Game(["Alice", "Bob"]); len(g.players)	assert len(g.players) == 2; assert g.players[0].name == "Alice"; assert g.players[1].name == "Bob"	PASSED (assertions satisfied)	tests/ test_game.py:: TestGameInit:: test_creates_players
Validate initializes state.	Game(["Alice", "Bob"])	assert g.state["current_index"] == 0; assert g.state["turn_number"] == 0; assert g.state["running"]	PASSED (assertions satisfied)	tests/ test_game.py:: TestGameInit:: test_initializes_state
Validate creates board.	Game(["Alice", "Bob"])	assert g.board is not None	PASSED (assertions satisfied)	tests/ test_game.py:: TestGameInit:: test_creates_board
Validate creates bank.	Game(["Alice", "Bob"])	assert g.bank is not None	PASSED (assertions satisfied)	tests/ test_game.py:: TestGameInit:: test_creates_bank

Continued on next page

Purpose/Justification	Input	Expected Output	Actual Output	Test Ref
Validate creates decks.	Game(["Alice", "Bob"])	assert g.decks["chance"] is not None; assert g.decks["community_chest"] is not None	PASSED (assertions satisfied)	tests/test_game.py::TestGameInit::test_creates_decks
Validate current player.	Game(["Alice", "Bob"]); g.current_player()	assert g.current_player().name == "Alice"	PASSED (assertions satisfied)	tests/test_game.py::TestGameTurnManager::test_current_player
Validate advance turn.	Game(["Alice", "Bob"]); g.advance_turn(); g.current_player()	assert g.current_player().name == "Bob"; assert g.state["turn_number"] == 1	PASSED (assertions satisfied)	tests/test_game.py::TestGameTurnManager::test_advance_turn
Validate advance turn wraps.	Game(["Alice", "Bob"]); g.advance_turn(); g.current_player()	assert g.current_player().name == "Alice"	PASSED (assertions satisfied)	tests/test_game.py::TestGameTurnManager::test_advance_turn_wrap
Validate successful purchase.	Game(["Alice", "Bob"]); g.current_player(); g.board.get_property_at(1)	assert result; assert prop.owner == player; assert player.balance == STARTING_BALANCE - 60; assert prop in player.properties	PASSED (assertions satisfied)	tests/test_game.py::TestGameBuyProperty::test_successful_purchase
Validate cannot afford.	Game(["Alice", "Bob"]); g.current_player(); g.board.get_property_at(1)	assert not result; assert prop.owner is None	PASSED (assertions satisfied)	tests/test_game.py::TestGameBuyProperty::test_cannot_afford
Validate exact balance can buy.	Game(["Alice", "Bob"]); g.current_player(); g.board.get_property_at(1)	assert result; assert player.balance == 0	PASSED (assertions satisfied)	tests/test_game.py::TestGameBuyProperty::test_exact_balance_can_
Validate pays rent to owner.	Game(["Alice", "Bob"]); g.board.get_property_at(1); bob.add_property(prop)	assert alice.balance == initial_alice - 2; assert bob.balance == initial_bob + 2	PASSED (assertions satisfied)	tests/test_game.py::TestGamePayRent::test_pays_rent_to_owner
Validate no rent on mortgaged.	Game(["Alice", "Bob"]); g.board.get_property_at(1); prop.mortgage()	assert alice.balance == initial	PASSED (assertions satisfied)	tests/test_game.py::TestGamePayRent::test_no_rent_on_mortga
Validate no rent if unowned.	Game(["Alice", "Bob"]); g.board.get_property_at(1); g.pay_rent(alice, prop)	assert alice.balance == initial	PASSED (assertions satisfied)	tests/test_game.py::TestGamePayRent::test_no_rent_if_unowned
Validate mortgage property.	Game(["Alice", "Bob"]); g.board.get_property_at(1); player.add_property(prop)	assert result; assert prop.is_mortgaged; assert player.balance == initial + 30; assert g.bank.get_balance() == initial_bank - 30	PASSED (assertions satisfied)	tests/test_game.py::TestGameMortgage::test_mortgage_property
Validate cannot mortgage others property.	Game(["Alice", "Bob"]); g.board.get_property_at(1); g.mortgage_property(alice, prop)	assert not result	PASSED (assertions satisfied)	tests/test_game.py::TestGameMortgage::test_cannot_mortgage_o
Validate cannot mortgage already mortgaged.	Game(["Alice", "Bob"]); g.board.get_property_at(1); player.add_property(prop)	assert not result	PASSED (assertions satisfied)	tests/test_game.py::TestGameMortgage::test_cannot_mortgage_a

Continued on next page

Purpose/Justification	Input	Expected Output	Actual Output	Test Ref
Regression for bug 11: mortgage flow should not call collect with negatives.	Game(["Alice", "Bob"]); g.board.get_property_at(1); player.add_property(prop)	Regression for bug 11: mortgage flow should not call collect with negatives.; assert call.args[0] ¿= 0	PASSED (assertions satisfied)	tests/ test_game.py:: TestGameMortgage:: test_mortgage_does_not
Validate unmortgage property.	Game(["Alice", "Bob"]); g.board.get_property_at(1); player.add_property(prop)	assert result; assert not prop.is_mortgaged; assert player.balance == initial - 33	PASSED (assertions satisfied)	tests/ test_game.py:: TestGameUnmortgage: test_unmortgage_proper
Validate cannot unmortgage not owned.	Game(["Alice", "Bob"]); g.board.get_property_at(1); prop.mortgage()	assert not result	PASSED (assertions satisfied)	tests/ test_game.py:: TestGameUnmortgage: test_cannot_unmortgage
Validate cannot afford unmortgage.	Game(["Alice", "Bob"]); g.board.get_property_at(1); player.add_property(prop)	assert not result; assert prop.is_mortgaged	PASSED (assertions satisfied)	tests/ test_game.py:: TestGameUnmortgage: test_cannot_afford_unm
Validate successful trade.	Game(["Alice", "Bob"]); g.board.get_property_at(1); alice.add_property(prop)	assert result; assert prop.owner == bob; assert prop in bob.properties; assert prop not in alice.properties	PASSED (assertions satisfied)	tests/ test_game.py:: TestGameTrade:: test_successful_trade
Validate trade fails not owner.	Game(["Alice", "Bob"]); g.board.get_property_at(1); g.trade(alice, bob, prop, 100)	assert not result	PASSED (assertions satisfied)	tests/ test_game.py:: TestGameTrade:: test_trade_fails_not_own
Validate trade fails cannot afford.	Game(["Alice", "Bob"]); g.board.get_property_at(1); alice.add_property(prop)	assert not result	PASSED (assertions satisfied)	tests/ test_game.py:: TestGameTrade:: test_trade_fails_cannot_
Validate find winner highest net worth.	Game(["Alice", "Bob"]); g.find_winner()	assert winner.name == "Bob"	PASSED (assertions satisfied)	tests/ test_game.py:: TestGameFindWinner:: test_find_winner_highes
Validate find winner with properties.	Game(["Alice", "Bob"]); g.board.get_property_at(39); bob.add_property(prop)	assert winner.name == "Bob"	PASSED (assertions satisfied)	tests/ test_game.py:: TestGameFindWinner:: test_find_winner_with_p
Validate find winner no players.	Game(["Alice", "Bob"]); g.players.clear(); g.find_winner()	assert g.find_winner() is None	PASSED (assertions satisfied)	tests/ test_game.py:: TestGameFindWinner:: test_find_winner_no_pla
Validate check bankruptcy eliminates player.	Game(["Alice", "Bob"]); g._check_bankruptcy(alice)	assert alice not in g.players; assert alice.is_eliminated	PASSED (assertions satisfied)	tests/ test_game.py:: TestGameBankruptcy:: test_check_bankruptcy_
Validate bankruptcy releases properties.	Game(["Alice", "Bob"]); g.board.get_property_at(1); alice.add_property(prop)	assert prop.owner is None; assert not prop.is_mortgaged	PASSED (assertions satisfied)	tests/ test_game.py:: TestGameBankruptcy:: test_bankruptcy_release
Validate collect card.	Game(["Alice", "Bob"]); g.current_player(); g._apply_card(player, card)	assert player.balance == initial + 50	PASSED (assertions satisfied)	tests/ test_game.py:: TestGameApplyCard:: test_collect_card
Validate pay card.	Game(["Alice", "Bob"]); g.current_player(); g._apply_card(player, card)	assert player.balance == initial - 50	PASSED (assertions satisfied)	tests/ test_game.py:: TestGameApplyCard:: test_pay_card

Continued on next page

Purpose/Justification	Input	Expected Output	Actual Output	Test Ref
Validate jail card.	Game(["Alice", "Bob"]); g.current_player(); g._apply_card(player, card)	assert player.jail_info["in_jail"]	PASSED (assertions satisfied)	tests/test_game.py::TestGameApplyCard::test_jail_card
Validate jail free card.	Game(["Alice", "Bob"]); g.current_player(); g._apply_card(player, card)	assert player.jail_info["get_out_of_jail_cards"] == 1	PASSED (assertions satisfied)	tests/test_game.py::TestGameApplyCard::test_jail_free_card
Validate null card.	Game(["Alice", "Bob"]); g.current_player(); g._apply_card(player, None)	assert player.balance == initial	PASSED (assertions satisfied)	tests/test_game.py::TestGameApplyCard::test_null_card
Regression for bug 12: low-balance players should still pay birthday fee.	Game(["Alice", "Bob", "Cara"]); g._apply_card(alice, card)	Regression for bug 12: low-balance players should still pay birthday fee.; assert bob.balance == -40; assert cara.balance == 50; assert alice.balance == STAR...	PASSED (assertions satisfied)	tests/test_game.py::TestGameApplyCard::test_birthday_card_colle
Regression for bug 16: players bankrupted by card effects should be removed immediately.	Game(["Alice", "Bob", "Cara"]); g._apply_card(alice, card)	Regression for bug 16: players bankrupted by card effects should be removed immediately.; assert bob not in g.players	PASSED (assertions satisfied)	tests/test_game.py::TestGameApplyCard::test_birthday_card_trigg
Validate landing on income tax.	patch("moneypoly.game.ui.confirm", return_value=False); patch("builtins.input", return_value="s"); Game(["Alice", "Bob"])	assert player.balance == initial - INCOME_TAX_AMOUNT	PASSED (assertions satisfied)	tests/test_game.py::TestGameMoveAndRes test_landing_on_income
Validate landing on go to jail.	patch("moneypoly.game.ui.confirm", return_value=False); patch("builtins.input", return_value="s"); Game(["Alice", "Bob"])	assert player.jail_info["in_jail"]	PASSED (assertions satisfied)	tests/test_game.py::TestGameMoveAndRes test_landing_on_go_to_ja
Regression for bug 14: paying to leave jail must reduce player balance.	patch("moneypoly.game.ui.confirm", return_value=True); patch("moneypoly.game.Dice.roll", return_value=0); patch("moneypoly.game.Game._mov...	Regression for bug 14: paying to leave jail must reduce player balance.; assert player.balance == initial_player - JAIL_FINE; assert g.bank.get_balance() == ...	PASSED (assertions satisfied)	tests/test_game.py::TestGameJailAndTurn test_voluntary_jail_fine_
Regression for bug 15: removing current player should not skip next player.	Game(["P1", "P2", "P3", "P4"]); g._check_bankruptcy(eliminated); g.advance_turn()	Regression for bug 15: removing current player should not skip next player.; assert g.current_player().name == "P3"	PASSED (assertions satisfied)	tests/test_game.py::TestGameJailAndTurn test_bankruptcy_during
Validate default balance.	Player("Alice")	assert p.balance == STARTING_BALANCE	PASSED (assertions satisfied)	tests/test_player.py::TestPlayerInit::test_default_balance
Validate custom balance.	Player("Bob", balance=500)	assert p.balance == 500	PASSED (assertions satisfied)	tests/test_player.py::TestPlayerInit::test_custom_balance

Continued on next page

Purpose/Justification	Input	Expected Output	Actual Output	Test Ref
Validate initial position.	Player("Alice")	assert p.position == 0	PASSED (assertions satisfied)	tests/ test_player.py:: TestPlayerInit:: test_initial_position
Validate initial state.	Player("Alice")	assert not p.jail_info["in_jail"]; assert p.jail_info["jail_turns"] == 0; assert p.jail_info["get_out_of_jail_cards"] == 0; assert not p.is_eliminated	PASSED (assertions satisfied)	tests/ test_player.py:: TestPlayerInit:: test_initial_state
Validate add money.	Player("Alice", balance=100); p.add_money(50)	assert p.balance == 150	PASSED (assertions satisfied)	tests/ test_player.py:: TestPlayerMoney:: test_add_money
Validate add zero.	Player("Alice", balance=100); p.add_money(0)	assert p.balance == 100	PASSED (assertions satisfied)	tests/ test_player.py:: TestPlayerMoney:: test_add_zero
Validate add negative raises.	Player("Alice"); p.add_money(-10)	Expect exception via pytest.raises(ValueError)	PASSED (assertions satisfied)	tests/ test_player.py:: TestPlayerMoney:: test_add_negative_raises
Validate deduct money.	Player("Alice", balance=100); p.deduct_money(30)	assert p.balance == 70	PASSED (assertions satisfied)	tests/ test_player.py:: TestPlayerMoney:: test_deduct_money
Validate deduct zero.	Player("Alice", balance=100); p.deduct_money(0)	assert p.balance == 100	PASSED (assertions satisfied)	tests/ test_player.py:: TestPlayerMoney:: test_deduct_zero
Validate deduct negative raises.	Player("Alice"); p.deduct_money(-10)	Expect exception via pytest.raises(ValueError)	PASSED (assertions satisfied)	tests/ test_player.py:: TestPlayerMoney:: test_deduct_negative_ra
Validate deduct more than balance.	Player("Alice", balance=50); p.deduct_money(100)	assert p.balance == -50	PASSED (assertions satisfied)	tests/ test_player.py:: TestPlayerMoney:: test_deduct_more_than_
Validate not bankrupt positive balance.	Player("Alice", balance=100); p.is_bankrupt()	assert not p.is_bankrupt()	PASSED (assertions satisfied)	tests/ test_player.py:: TestPlayerBankruptcy: test_not_bankrupt_posit
Validate bankrupt at zero.	Player("Alice", balance=0); p.is_bankrupt()	assert p.is_bankrupt()	PASSED (assertions satisfied)	tests/ test_player.py:: TestPlayerBankruptcy: test_bankrupt_at_zero
Validate bankrupt negative.	Player("Alice", balance=-50); p.is_bankrupt()	assert p.is_bankrupt()	PASSED (assertions satisfied)	tests/ test_player.py:: TestPlayerBankruptcy: test_bankrupt_negative
Validate basic move.	Player("Alice"); p.move(5)	assert pos == 5; assert p.position == 5	PASSED (assertions satisfied)	tests/ test_player.py:: TestPlayerMovement:: test_basic_move
Validate wrap around board.	Player("Alice"); p.move(4)	assert pos == 2; assert p.balance == STARTING_BALANCE + GO_SALARY	PASSED (assertions satisfied)	tests/ test_player.py:: TestPlayerMovement:: test_wrap_around_board

Continued on next page

Purpose/Justification	Input	Expected Output	Actual Output	Test Ref
Validate land on go.	Player("Alice"); p.move(5)	assert pos == 0; assert p.balance == STARTING_BALANCE + GO_SALARY	PASSED (assertions satisfied)	tests/ test_player.py:: TestPlayerMovement:: test_land_on_go
Validate no go salary without passing.	Player("Alice"); p.move(3)	assert p.balance == STARTING_BALANCE	PASSED (assertions satisfied)	tests/ test_player.py:: TestPlayerMovement:: test_no_go_salary_witho
Validate move zero steps.	Player("Alice"); p.move(0)	assert pos == 10	PASSED (assertions satisfied)	tests/ test_player.py:: TestPlayerMovement:: test_move_zero_steps
Validate go to jail.	Player("Alice"); p.go_to_jail()	assert p.position == JAIL_POSITION; assert p.jail_info["in_jail"]; assert p.jail_info["jail_turns"] == 0	PASSED (assertions satisfied)	tests/ test_player.py:: TestPlayerJail:: test_go_to_jail
Validate go to jail resets turns.	Player("Alice"); p.go_to_jail()	assert p.jail_info["jail_turns"] == 0	PASSED (assertions satisfied)	tests/ test_player.py:: TestPlayerJail:: test_go_to_jail_resets_tu
Validate add property.	Player("Alice"); p.add_property("prop1")	assert "prop1" in p.properties	PASSED (assertions satisfied)	tests/ test_player.py:: TestPlayerProperties:: test_add_property
Validate add duplicate property.	Player("Alice"); p.add_property("prop1"); len(p.properties)	assert len(p.properties) == 1	PASSED (assertions satisfied)	tests/ test_player.py:: TestPlayerProperties:: test_add_duplicate_prop
Validate remove property.	Player("Alice"); p.add_property("prop1"); p.remove_property("prop1")	assert "prop1" not in p.properties	PASSED (assertions satisfied)	tests/ test_player.py:: TestPlayerProperties:: test_remove_property
Validate remove nonexistent property.	Player("Alice"); p.remove_property("prop1")	Test should complete without error for given inputs.	PASSED (assertions satisfied)	tests/ test_player.py:: TestPlayerProperties:: test_remove_nonexisten
Validate count properties.	Player("Alice"); p.add_property("a"); p.add_property("b")	assert p.count_properties() == 0; assert p.count_properties() == 2	PASSED (assertions satisfied)	tests/ test_player.py:: TestPlayerProperties:: test_count_properties
Validate net worth no properties.	Player("Alice", balance=500); p.net_worth()	assert p.net_worth() == 500	PASSED (assertions satisfied)	tests/ test_player.py:: TestPlayerNetWorth:: test_net_worth_no_prop
Validate net worth with properties.	Player("Alice", balance=500); Property("Test", 1, {"price": 200, "base_rent": 10}); p.add_property(prop)	assert p.net_worth() == 700	PASSED (assertions satisfied)	tests/ test_player.py:: TestPlayerNetWorth:: test_net_worth_with_pro
Validate status line.	Player("Alice"); p.status_line(); str(STARTING_BALANCE)	assert "Alice" in line; assert str(STARTING_BALANCE) in line	PASSED (assertions satisfied)	tests/ test_player.py:: TestPlayerDisplay:: test_status_line
Validate status line jailed.	Player("Alice"); p.status_line()	assert "[JAILED]" in p.status_line()	PASSED (assertions satisfied)	tests/ test_player.py:: TestPlayerDisplay:: test_status_line_jailed
Validate repr.	Player("Alice"); repr(p)	assert "Alice" in repr(p)	PASSED (assertions satisfied)	tests/ test_player.py:: TestPlayerDisplay:: test_repr

Continued on next page

Purpose/Justification	Input	Expected Output	Actual Output	Test Ref
Validate basic creation.	Property("Test Ave", 1, {"price": 100, "base_rent": 10})	assert p.name == "Test Ave"; assert p.position == 1; assert p.economics["price"] == 100; assert p.economics["base_rent"] == 10	PASSED (assertions satisfied)	tests/test_property.py::TestPropertyInit::test_basic_creation
Validate with group.	PropertyGroup("Brown", "brown"); Property("Test Ave", 1, {"price": 100, "base_rent": 10}, group=g)	assert p.group == g; assert p in g.properties	PASSED (assertions satisfied)	tests/test_property.py::TestPropertyInit::test_with_group
Validate no duplicate group registration.	PropertyGroup("Brown", "brown"); Property("Test Ave", 1, {"price": 100, "base_rent": 10}, group=g); Property("Test2", 2, {"price": 100, "...	assert len(g.properties) == 2	PASSED (assertions satisfied)	tests/test_property.py::TestPropertyInit::test_no_duplicate_group
Validate base rent.	Property("Test Ave", 1, {"price": 100, "base_rent": 10}); Player("Alice"); p.get_rent()	assert p.get_rent() == 10	PASSED (assertions satisfied)	tests/test_property.py::TestPropertyRent::test_base_rent
Validate rent doubled with full group.	PropertyGroup("Brown", "brown"); Property("Ave1", 1, {"price": 60, "base_rent": 2}, group=g); Property("Ave2", 3, {"price": 60, "base_ren...	assert p1.get_rent() == 4; assert p2.get_rent() == 8	PASSED (assertions satisfied)	tests/test_property.py::TestPropertyRent::test_rent_doubled_with_
Validate rent not doubled partial group.	PropertyGroup("Brown", "brown"); Property("Ave1", 1, {"price": 60, "base_rent": 2}, group=g); Property("Ave2", 3, {"price": 60, "base_ren...	assert p1.get_rent() == 2	PASSED (assertions satisfied)	tests/test_property.py::TestPropertyRent::test_rent_not_doubled_p
Validate rent zero when mortgaged.	Property("Test Ave", 1, {"price": 100, "base_rent": 10}); Player("Alice"); p.mortgage()	assert p.get_rent() == 0	PASSED (assertions satisfied)	tests/test_property.py::TestPropertyRent::test_rent_zero_when_mo
Validate mortgage returns half price.	Property("Test Ave", 1, {"price": 100, "base_rent": 10}); p.mortgage()	assert payout == 50; assert p.is_mortgaged	PASSED (assertions satisfied)	tests/test_property.py::TestPropertyMortgage::test_mortgage_returns_h
Validate mortgage already mortgaged.	Property("Test Ave", 1, {"price": 100, "base_rent": 10}); p.mortgage()	assert p.mortgage() == 0	PASSED (assertions satisfied)	tests/test_property.py::TestPropertyMortgage::test_mortgage_already_r
Validate unmortgage returns cost.	Property("Test Ave", 1, {"price": 100, "base_rent": 10}); p.mortgage(); p.unmortgage()	assert cost == 55; assert not p.is_mortgaged	PASSED (assertions satisfied)	tests/test_property.py::TestPropertyMortgage::test_unmortgage_return
Validate unmortgage not mortgaged.	Property("Test Ave", 1, {"price": 100, "base_rent": 10}); p.unmortgage()	assert p.unmortgage() == 0	PASSED (assertions satisfied)	tests/test_property.py::TestPropertyMortgage::test_unmortgage_not_m
Validate is available.	Property("Test Ave", 1, {"price": 100, "base_rent": 10}); p.is_available(); Player("Alice")	assert p.is_available(); assert not p.is_available()	PASSED (assertions satisfied)	tests/test_property.py::TestPropertyMortgage::test_is_available
Validate is available mortgaged.	Property("Test Ave", 1, {"price": 100, "base_rent": 10}); p.is_available()	assert not p.is_available()	PASSED (assertions satisfied)	tests/test_property.py::TestPropertyMortgage::test_is_available_mortga

Continued on next page

Purpose/Justification	Input	Expected Output	Actual Output	Test Ref
Validate all owned by single owner.	PropertyGroup("Brown", "brown"); Property("A", 1, {"price": 60, "base_rent": 2}, group=g); Property("B", 3, {"price": 60, "base_rent": 4}...	assert g.all_owned_by(alice)	PASSED (assertions satisfied)	tests/ test_property.py:: TestPropertyGroup:: test_all_owned_by_single
Validate all owned by fails with mixed owners.	PropertyGroup("Brown", "brown"); Property("A", 1, {"price": 60, "base_rent": 2}, group=g); Property("B", 3, {"price": 60, "base_rent": 4}...	assert not g.all_owned_by(alice); assert not g.all_owned_by(bob)	PASSED (assertions satisfied)	tests/ test_property.py:: TestPropertyGroup:: test_all_owned_by_fails_
Validate all owned by none returns false.	PropertyGroup("Brown", "brown"); Property("A", 1, {"price": 60, "base_rent": 2}, group=g); g.all_owned_by(None)	assert not g.all_owned_by(None)	PASSED (assertions satisfied)	tests/ test_property.py:: TestPropertyGroup:: test_all_owned_by_none_
Validate all owned by partial unowned.	PropertyGroup("Brown", "brown"); Property("A", 1, {"price": 60, "base_rent": 2}, group=g); Property("B", 3, {"price": 60, "base_rent": 4}...	assert not g.all_owned_by(alice)	PASSED (assertions satisfied)	tests/ test_property.py:: TestPropertyGroup:: test_all_owned_by_parti
Validate get owner counts.	PropertyGroup("Brown", "brown"); Property("A", 1, {"price": 60, "base_rent": 2}, group=g); Property("B", 3, {"price": 60, "base_rent": 4}...	assert counts[alice] == 2	PASSED (assertions satisfied)	tests/ test_property.py:: TestPropertyGroup:: test_get_owner_counts
Validate size.	PropertyGroup("Brown", "brown"); Property("A", 1, {"price": 60, "base_rent": 2}, group=g); Property("B", 3, {"price": 60, "base_rent": 4}...	assert g.size() == 2	PASSED (assertions satisfied)	tests/ test_property.py:: TestPropertyGroup:: test_size
Validate add property.	PropertyGroup("Brown", "brown"); Property("A", 1, {"price": 60, "base_rent": 2}); g.add_property(p)	assert p in g.properties; assert p.group == g	PASSED (assertions satisfied)	tests/ test_property.py:: TestPropertyGroup:: test_add_property
Validate repr.	PropertyGroup("Brown", "brown"); repr(g)	assert "Brown" in repr(g)	PASSED (assertions satisfied)	tests/ test_property.py:: TestPropertyGroup:: test_repr
Validate property repr.	Property("Test", 1, {"price": 100, "base_rent": 10}); Player("Alice"); repr(p)	assert "unowned" in repr(p); assert "Alice" in repr(p)	PASSED (assertions satisfied)	tests/ test_property.py:: TestPropertyGroup:: test_property_repr

5 Conclusion

To summarize, the core engine underwent rigorous white-box testing and iterative code-quality refinement. By systematically addressing Pylint warnings—targeting dormant imports, convoluted branch logic, and missing documentation—the codebase achieved perfect compliance. Furthermore, resolving the 16 hidden logical flaws ensures a robust, mathematically sound execution of the game rules, guaranteeing an error-free simulation without any remaining structural smells.